

并行与分布式计算作业2

- 姓名：张昕跃
- 学号：2023202300
- 班级：2023级图灵实验班

思考 1

问题：CUDA C/C++ 关键字 `__global__` 表示什么？`<<< >>>` 表示什么？

答：

`__global__` 是一个函数声明，表示是一个 GPU kernel，由Host调用（cpu）、在device上执行（gpu）。

`nvcc`编译器会将host和device代码进行区分，Device函数（`mykernel()`）由NVIDIA编译器编译，Host函数（`main()`）由标准host端编译器（`gcc`）编译

对于例如 `kernel<<<N,M>>>();`，是一个kernel launch，表示host调用device端的代码。开启N个block（grid的维度），每个block有M个线程（block的维度）

思考 2

问题：`cudaMalloc()`、`cudaFree()`、`cudaMemcpy()` 分别表示什么意思？

答：

类似 C 中的 `malloc()`, `free()`, `memcpy()`。

`cudaMalloc()` 即在GPU全局内存上申请一段连续空间；

`cudaFree()` 释放之前申请的内存；

`cudaMemcpy()` 在主机和设备之间或设备内部执行内存拷贝。

方向由第四个参数指定（`cudaMemcpyHostToDevice` 表示把inputs copy to gpu device，`cudaMemcpyDeviceToHost` 表示copy results back to host cpu）。

思考 3

问题：调用 GPU kernel `add`，打算开 32 个 block，每个 block 里有 64 个线程，请问 `main` 函数里怎么写？

答：

注意`add`函数需要加 `__global__` 修饰。要先分配内存，copy数据到cpu再计算，最后还要把结果copy回来。

```
__global__ void add(int *a, int *b, int *c)
{
    *c = *a + *b;
}

int main()
```

```

{
    int blockNum = 32;
    int threadNum = 64;

    // host copies of a, b, c
    int a, b, out;
    // device copies of a, b, c
    int *d_a, *d_b, *d_out;

    int size = sizeof(int);
    // Allocate space for device copies of a, b, out
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_out, size);

    // // Setup input values
    // a = 2;
    // b = 7;

    // Copy inputs to device
    cudaMemcpy(d_a, &a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, &b, size, cudaMemcpyHostToDevice);
    // Launch add() kernel on GPU
    add<<<blockNum, threadNum>>>(d_a, d_b, d_c);

    // Copy result back to host
    cudaMemcpy(&c, d_c, size, cudaMemcpyDeviceToHost);

    // Cleanup
    cudaFree(d_a);
    cudaFree(d_b);
    cudaFree(d_out);
    return 0;
}

```

思考 4

问题：为什么要有 shared memory?

答：

同一 block 内部，线程可以通过 shared memory 访问共享数据。

shared memory 位于每个 SM 上，访问延迟远低于全局内存，可以减少对全局内存的重复访问，实现线程间快速数据交换，从而提高访存吞吐和整体性能。

思考 5

问题：已知 GPU 型号为 GeForce RTX 2080 Ti，计算能力 7.5，请问对于 `main.cu`，编译命令怎么写？

答：

```
nvcc -arch=sm_75 main.cu -o main
# nvcc main.cu -o main \
#     直接使用
#     -gencode=arch=compute_75,code=sm_75 \
#     前向兼容更高等级cpu
#     -gencode=arch=compute_75,code=compute_75
```